Sistem preporuka u aplikaciji Nestly

1. Opis sistema preporuka

Pametni sistem preporuka implementiran u aplikaciji Nestly temelji se na personalizovanom modelu zasnovanom na ponašanju korisnika i kontekstualnim podacima o fazi roditeljstva. Implementiran je kao sistem preporuke za blogove koji analizira prethodne korisničke interakcije i prilagođava sadržaj individualnim potrebama korisnika.

Cilj sistema je da korisnicima prikaže najrelevantnije blog članke u skladu sa njihovom trenutnom životnom fazom (trudnoća ili period nakon rođenja djeteta), starošću bebe ili sedmicom trudnoće, kao i interesovanjima iskazanim kroz prethodno čitanje sadržaja.

Za razliku od klasičnih sistema koji koriste isključivo popularnost ili statičke filtre, Nestly sistem kombinuje više faktora:

- Fazu korisnika (BellyTime ili BabyTime)
- Trenutnu sedmicu trudnoće ili starost bebe u sedmicama
- Kategorije sadržaja koje korisnik najčešće čita
- Globalnu popularnost članka
- Dinamičko učenje putem online treniranja modela

Sistem koristi pojednostavljeni model logističke regresije sa sigmoid funkcijom za izračun vjerovatnoće relevantnosti svakog članka. Težine modela (weights) čuvaju se u bazi podataka i ažuriraju se dinamički na osnovu korisničkih interakcija, čime sistem kontinuirano uči i prilagođava se ponašanju korisnika.

2. Model podataka za praćenje interakcija

Putanja:
Nestly-backend/Nestly.Model/Entity/BlogPostInteraction.cs

```
using Nestly.Model.DTOObjects;
using System;
using System.ComponentModel.DataAnnotations;

namespace Nestly.Model.Entity
{
  public class BlogPostInteraction
  {
    [Key]
    public long Id { get; set; }
    public long UserId { get; set; }
    public long PostId { get; set; }
    public ArticleEventType EventType { get; set; }
    public int? SpentSeconds { get; set; }
    public DateTime CreatedAt { get; set; }
  }
```

```
}
```

Enum definicije:

Putanja:
Nestly-backend/Nestly.Model/DTOObjects/UserPhase.cs

```
namespace Nestly.Model.DTOObjects
{
  public enum UserPhase
  {
    BellyTime = 1,
    BabyTime = 2
  }
  public enum ArticleEventType
  {
    View = 1,
    Open = 2,
  }
}
```

3. DTO objekti sistema preporuka

Putanja:
Nestly-backend/Nestly.Model/DTOObjects/BlogRecommendationDto.cs

```
namespace Nestly.Model.DTOObjects
{
  public class RecommendedBlogPostResponse
  {
    public long PostId { get; set; }
    public string Title { get; set; }
    public string? ImageUrl { get; set; }
    public double Score { get; set; }
  }
  public class LogBlogInteractionRequest
  {
    public long PostId { get; set; }
    public int EventType { get; set; }
    public int? SpentSeconds { get; set; }
  }
}
```

4. Glavna logika servisa preporuka

Putanja:

Nestly-backend/Nestly.Services/Repository/BlogRecommendationService.cs

```
using Microsoft.EntityFrameworkCore;
using Nestly.Model.DTOObjects;
using Nestly.Model.Entity;
using Nestly.Services.Data;
using Nestly.Services.Interfaces;
using System.Text.Json;

namespace Nestly.Services.Repository
{
  public class BlogRecommendationService : IBlogRecommendationService
  {
    private readonly NestlyDbContext _db;

    public BlogRecommendationService(NestlyDbContext db)
    {
      _db = db;
    }

    public async Task<List<BlogPostResponseDto>> GetRecommendations(long userId, int
take)
    {
      var context = await BuildUserContext(userId);
      var weights = await GetWeights();

      var posts = await _db.BlogPosts
        .Include(p => p.BlogPostCategories)
        .ThenInclude(pc => pc.Category)
        .Where(p => p.Phase == context.Phase)
        .ToListAsync();

      var scored = new List<(BlogPost post, double score)>();

      foreach (var post in posts)
      {
        double weekScore = ComputeWeekScore(context.WeekValue, post.WeekFrom,
post.WeekTo);
        double categoryAffinity = await ComputeCategoryAffinity(userId, post.Id);
        double popularity = await ComputePopularity(post.Id);

        var features = new Dictionary<string, double>
    {
```

```
    { "bias", 1.0 },
    { "weekScore", weekScore },
    { "categoryAffinity", categoryAffinity },
    { "popularity", popularity }
};

        double score = Sigmoid(Dot(weights, features));

        scored.Add((post, score));
    }

    return scored
        .OrderByDescending(x => x.score)
        .Take(take)
        .Select(x => new BlogPostResponseDto
        {
            Id = x.post.Id,
            Title = x.post.Title,
            Content = x.post.Content,
            ImageUrl = x.post.ImageUrl,
            AuthorId = x.post.AuthorId,
            Phase = x.post.Phase,
            WeekFrom = x.post.WeekFrom,
            WeekTo = x.post.WeekTo
        })
        .ToList();
}


public async Task LogInteraction(long userId, LogBlogInteractionRequest request)
{
    _db.BlogPostInteractions.Add(new BlogPostInteraction
    {
        UserId = userId,
        PostId = request.PostId,
        EventType = (ArticleEventType)request.EventType,
        SpentSeconds = request.SpentSeconds,
        CreatedAt = DateTime.UtcNow
    });

    await _db.SaveChangesAsync();

    bool isPositive = request.EventType == 2 && request.SpentSeconds >= 15;

    await Train(userId, request.PostId, isPositive);
}

private async Task Train(long userId, long postId, bool positive)
```

```
    {
        var weights = await GetWeights();
        var context = await BuildUserContext(userId);
        var post = await _db.BlogPosts.FirstAsync(p => p.Id == postId);

        double weekScore = ComputeWeekScore(context.WeekValue, post.WeekFrom,
post.WeekTo);
        double categoryAffinity = await ComputeCategoryAffinity(userId, post.Id);
        double popularity = await ComputePopularity(post.Id);

        var features = new Dictionary<string, double>
        {
            { "bias", 1.0 },
            { "weekScore", weekScore },
            { "categoryAffinity", categoryAffinity },
            { "popularity", popularity }
        };

        double prediction = Sigmoid(Dot(weights, features));
        double target = positive ? 1 : 0;
        double error = target - prediction;
        double lr = 0.1;

        foreach (var f in features)
        {
            weights[f.Key] += lr * error * f.Value;
        }

        await SaveWeights(weights);
    }

    private async Task<(UserPhase Phase, int WeekValue)> BuildUserContext(long userId)
    {
        var user = await _db.AppUsers
            .Include(u => u.ParentProfile)
            .ThenInclude(p => p.Babies)
            .Include(u => u.ParentProfile)
            .ThenInclude(p => p.Pregnancies)
            .FirstAsync(u => u.Id == userId);

        if (user.ParentProfile?.Babies.Any() == true)
        {
            var baby = user.ParentProfile.Babies
                .OrderByDescending(b => b.BirthDate)
                .First();

            int weeks = (int)((DateTime.UtcNow - baby.BirthDate).TotalDays / 7);
```

```
        return (UserPhase.BabyTime, weeks);
    }

    if (user.ParentProfile?.Pregnancies.Any() == true)
    {
        var preg = user.ParentProfile.Pregnancies
            .OrderByDescending(p => p.DueDate)
            .First();

        int weeks = 40 - (int)((preg.DueDate.Value - DateTime.UtcNow).TotalDays / 7);

        return (UserPhase.BellyTime, weeks);
    }

    return (UserPhase.BellyTime, 1);
}

private double ComputeWeekScore(int userWeek, int? from, int? to)
{
    if (!from.HasValue || !to.HasValue)
    {
        return 0.5;
    }

    if (userWeek >= from && userWeek <= to)
    {
        return 1.0;
    }

    int distance = userWeek < from ? from.Value - userWeek : userWeek - to.Value;

    return 1.0 / (1 + distance);
}

private async Task<double> ComputeCategoryAffinity(long userId, long postId)
{
    var postCategories = await _db.BlogPostCategories
        .Where(pc => pc.PostId == postId)
        .Select(pc => pc.CategoryId)
        .ToListAsync();

    var interactions = await _db.BlogPostInteractions
        .Where(i => i.UserId == userId)
        .Join(_db.BlogPostCategories,
            i => i.PostId,
            pc => pc.PostId,
            (i, pc) => new { i, pc })
        .Where(x => postCategories.Contains(x.pc.CategoryId))
```

```
        .CountAsync();

    return interactions / 10.0;
}

private async Task<double> ComputePopularity(long postId)
{
    int count = await _db.BlogPostInteractions
        .CountAsync(i => i.PostId == postId);

    return count / 50.0;
}

private async Task<Dictionary<string, double>> GetWeights()
{
    var state = await _db.RecommendationModelStates
        .OrderByDescending(x => x.Id)
        .FirstOrDefaultAsync();

    if (state == null)
    {
        var initial = new Dictionary<string, double>
{
    { "bias", 0 },
    { "weekScore", 1 },
    { "categoryAffinity", 1 },
    { "popularity", 0.5 }
};

        var newState = new RecommendationModelState
        {
            UpdatedAt = DateTime.UtcNow,
            WeightsJson = JsonSerializer.Serialize(initial)
        };

        _db.RecommendationModelStates.Add(newState);
        await _db.SaveChangesAsync();

        return initial;
    }

    return JsonSerializer.Deserialize<Dictionary<string, double>>(state.WeightsJson);
}

private async Task SaveWeights(Dictionary<string, double> weights)
{
    var state = await _db.RecommendationModelStates
        .OrderByDescending(x => x.Id)
```

```
            .FirstOrDefaultAsync();

        if (state == null)
        {
            state = new RecommendationModelState();
            _db.RecommendationModelStates.Add(state);
        }

        state.WeightsJson = JsonSerializer.Serialize(weights);
        state.UpdatedAt = DateTime.UtcNow;

        await _db.SaveChangesAsync();
    }


    private double Dot(Dictionary<string, double> w, Dictionary<string, double> x)
    {
        double sum = 0;
        foreach (var f in x)
        {
            sum += w[f.Key] * f.Value;
        }

        return sum;
    }

    private double Sigmoid(double z)
    {
        return 1.0 / (1.0 + Math.Exp(-z));
    }
  }
}
```

5. Kontroler sistema preporuka

Putanja:

Nestly-backend/Nestly/Controllers/RecommendationController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Nestly.Model.DTOObjects;
using Nestly.Services.Interfaces;
using System.Security.Claims;

namespace Nestly_WebAPI.Controllers
{
  [ApiController]
```

```
  [Route("api/recommendations")]
  [Authorize]
  public class RecommendationController : ControllerBase
  {
    private readonly IBlogRecommendationService _service;

    public RecommendationController(IBlogRecommendationService service)
    {
      _service = service;
    }

    [HttpGet("blog")]
    public async Task<IActionResult> Get(int take = 10)
    {
      var claim = User.FindFirst(ClaimTypes.NameIdentifier);

      if (claim == null)
      {
        return Unauthorized("User id claim not found.");
      }

      var userId = long.Parse(claim.Value);

      return Ok(await _service.GetRecommendations(userId, take));
    }

    [HttpPost("log")]
    public async Task<IActionResult> Log(LogBlogInteractionRequest req)
    {
      var claim = User.FindFirst(ClaimTypes.NameIdentifier);

      if (claim == null)
      {
        return Unauthorized("User id claim not found.");
      }

      var userId = long.Parse(claim.Value);

      await _service.LogInteraction(userId, req);
      return Ok();
    }
  }
}
```

6. Prikaz preporuka u aplikaciji

Putanja:

Mobilna aplikacija -> Blog